

作业感受（第二次挑战）

HW2 第五部分

（1）简述你是如何完成第二次挑战的

这一次的关键词是"对照"。我先按题目要求把实验口径定死：两组各 32 条请求、并发 8、输入固定 1024 token（共享前缀 896 + 各自后缀 128）、`max_new_tokens=16`、`temperature=0`、`ignore_eos=true`、`sampling_seed=2026`，除"前缀能否复用"外全部保持一致，然后用脚本把流程固化下来：服务预热 → 等待空闲 → `POST /flush_cache` 并校验返回 → 共享前缀组额外发一条预热请求 → 并发回放。采集指标时我特意区分了客户端计时（TTFT/TPOT/E2E）与服务端 `meta_info`（`cached_tokens`），这样"命中了多少"与"快了多少"能互相对照。

中途踩了三个坑，都是靠"让脚本自己报错"发现的：一是把 `/flush_cache` 的返回判错，脚本一直认为服务不空闲；二是第一版"分散前缀"其实生成了相同文本，分散组命中率高达 74.9%，我加了"最长公共前缀自检"才暴露出来，改成分散语料后降到 0.56%；三是容器内存限额只有 8 GB，KV 池开大了会被 OOM 杀掉。修完后两组各跑 3 轮，结果非常稳定（命中率三轮完全一致），才开始动笔写报告。AI 在其中主要扮演"报错解释器 + 源码导航 + 文档骨架"的角色，而实验设计、指标口径与结论判断都是我自己定的。

（2）你觉得哪部分工作最困难，你是如何克服这个困难的

这次最难的其实不是把服务跑起来，而是让两组负载真的只在"前缀能不能复用"这一点上不一样。第一版脚本我图省事，分散前缀组拿同一份语料循环拼接、只换起始位置，跑出来命中率 74.9%——也就是说我以为"分散"的那组，前缀几乎全一样。这个数字要是不看，报告照样写得出来，结论看着也挺顺，只是经不起推敲。后来我加了一条自检：把前几条请求两两算最长公共前缀长度打出来，共享组应该约等于 896、分散组应该约等于 0。一打就露馅了。改成分散语料、每个 seed 重新排列句子顺序之后，分散组命中率降到 0.56%。

还有一次也差不多：共享组平均命中 919.5 token，比我自己设计的前缀 896 还多。我第一反应是统计写错了，去翻 `radix_cache.py` 才明白——写回缓存的是**整条序列**（前缀加后缀），后面某条请求的 128 token 后缀要是和已缓存序列的后缀开头撞上了，最长前缀匹配就会再往前多咬一口。这种"数据不听话"的时刻反而是这次收获最大的地方：数字跟预期对不上时，多数不是脚本错了，是我对机制的理解还差一层。

（3）你觉得类似的从 0 到 1 的科研工作，需要包含哪些方面的研究，对你今后的科研工作有什么启发

要是把这次的过程抽象成"从 0 到 1 需要什么"，我会说四样东西。一是**一个真能跑起来的最小系统**，而且它的版本、启动参数、数据来源都能让别人照着复现出来；二是**可信的测量**，指标要在动手之前就定好，最好同时留服务端计数器（`cached_tokens`）和客户端计时（TTFT/TPOT/E2E）两套数，这样"命中了多少"和"真的快了多少"能互相印证，不至于被单方面的数字骗过去；三是**对照实验**，一次只动一个变量，而且要对负载本身的性质做断言（前缀到底分不分散），不能只看脚本有没有跑通；四是**机制层面的解释**，从注意力公式一路推到源码里具体的那一行，否则"提升 1.79 倍"就只是一个数字，换台机器可能就不成立了。

对我自己做研究的启发，主要是两点挺朴素的。一是别急着下结论：我看到 TTFT 降了、TPOT 没动，差点直接写成"TPOT 不受前缀复用影响"，回头看数据才发现这两者在纯 CPU 串行下是此消彼长的关系，跨组只能拿端到端时长和吞吐说话。二是别嫌"看起来没进度"的事慢——读源码、核对行号、给脚本加自检，当时都觉得在耽误时间，最后报告里能站得住的句子基本都来自那儿。AI 帮我省掉的主要是检索和试错的时间，至于哪些数字能信、哪些结论敢写，还是得自己判断。